```
policyBuilder => policyBuilder.RequireClaim("BoardingPassNumber", "A1234");
```

Table 24.1 Simple policy builder methods on `AuthorizationPolicyBuilder`

Method	Policy behavior
`RequireAuthenticatedUser()`	The required user must be authenticated. Creates a policy similar to the default `[Authorize]` attribute, where you don't set a policy.
`RequireClaim(claim, values)`	The user must have the specified claim. If provided, the claim must be one of the specified values.
`RequireUsername(username)`	The user must have the specified username.
`RequireAssertion(function)`	Executes the provided lambda function, which returns a `bool`, indicating whether the policy was satisfied.

Role-based authorization vs. claims-based authorization

If you look at all of the methods available on the `AuthorizationPolicyBuilder` type using IntelliSense, you might notice that there's a method I didn't mention in table 24.1: `RequireRole()`. This is a remnant of the role-based approach to authorization used in previous versions of ASP.NET, and I don't recommend using it.

Before Microsoft adopted the claims-based authorization used by ASP.NET, role-based authorization was the norm. Users were assigned to one or more roles, such as `Administrator` or `Manager`, and authorization involved checking whether the current user was in the required role.

This role-based approach to authorization is possible in ASP.NET Core, but it's used primarily for legacy compatibility reasons. Claims-based authorization is the suggested approach. Unless you're porting a legacy app that uses roles, I suggest that you embrace claims-based authorization and leave those roles behind.